Cyberscope

A TAC Security Company

Audit Report

UCBI Banking Token

May 2026

Network	ETH
Address	0xb42b35deca033a23401a1a89007a39343a510d0a
Audited by	© cyberscope

Analysis

Critical Medium Minor / Informative Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Acknowledged
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Acknowledged

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
●	CCR	Contract Centralization Risk	Acknowledged
●	MEE	Missing Events Emission	Acknowledged
●	MRO	Missing Renounce Ownership	Acknowledged
●	MTEE	Missing Transfer Event Emission	Acknowledged
●	L02	State Variables could be Declared Constant	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved
●	L19	Stable Compiler Version	Unresolved

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	UCBI Banking
Explorer	https://etherscan.io/address/0xb42b35deca033a23401a1a89007a39343a510d0a
Symbol	UCBI
Decimals	8
Total Supply	20,000,000

Audit Updates

Initial Audit	21 Apr 2026 https://github.com/cyberscope-io/audits/blob/main/ucbi/v1/audit.pdf
	22 May 2026

Source Files

Filename	SHA256
UCBIBanking.sol	6eb7c7db05a70d8cb03f652e6fa1f7c3cdf201224b20b32ce96ec32700cf4b57

Overview

1. Introduction

This overview documents the role of the UCBI token within the UCBI ecosystem as described in the project whitepaper. It is provided for contextual reference alongside the security audit of the `UCBIHolding` contract and reflects the team's intended approach as communicated through the whitepaper. No part of this overview constitutes an independent verification of the ecosystem or its intended functions.

---

2. The UCBI Platform

The whitepaper describes the UCBI platform's activities as including:

- The accumulation and management of a digital treasury centred on Ethereum.
- Allocation across additional asset classes, such as real estate.
- The delivery of treasury advisory services.
- Support services for third-party token launches, covering tokenomics design, liquidity structuring, launch strategy advice, preparation for listing, and cash flow sustainability planning.

Within this structure, the whitepaper presents the UCBI token as a functional component of the platform's technological and community ecosystem.

3. Role of the UCBI Token in the Ecosystem

The whitepaper lists the following functional roles for the UCBI token within the platform's ecosystem:

Role	Description
Platform utility	Use of the token in connection with features and services delivered through the UCBI platform.
Community incentives	Use of the token within community-oriented initiatives..
Staking participation	Use of the token in staking arrangements that the whitepaper describes as part of the intended ecosystem design.
Strategic ecosystem expansion	Use of the token in activities that contribute to extending the ecosystem over time.

The whitepaper additionally states that the token enhances liquidity and community engagement, and references governance potential as a direction subject to further review.

The whitepaper further characterises the token as a structural element of the broader UCBI ecosystem. The roles described above reflect the team's intended approach as communicated through the whitepaper.

4. Relationship to the Audited Contract

The `UCBIHolding` contract reviewed in the accompanying audit report provides the on-chain substrate on which the UCBI token operates.

At the contract level, this substrate consists of:

- A transferable ERC20 unit with a fixed total supply of 20,000,000 units and 8 decimals.
- A burn capability that allows holders to permanently destroy their own tokens.
- A centralized transfer-control mechanism that allows the contract owner to pause transfers globally or selectively restrict individual addresses through on-chain allowlist and blocklist mappings.

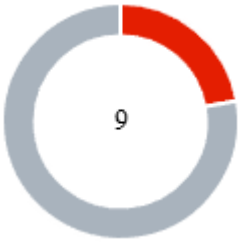
The ecosystem functions referenced in the whitepaper including platform utility accrual, community-incentive distribution, staking participation, governance arrangements, liquidity-enhancement mechanisms, and other forms of strategic ecosystem expansion — are not implemented within the audited contract. Their delivery would rely on additional smart contracts or off-chain processes that are not part of the reviewed codebase. This reflects the team's intended approach as described in the whitepaper.

---

5. Scope of this Overview

The content of this overview reflects the team's intended approach as communicated through the whitepaper. It is provided for contextual reference and does not represent an independent verification of the ecosystem functions, the platform's outlook, or the future implementation of any component described in the whitepaper.

Findings Breakdown



Critical	2
Medium	0
Minor / Informative	7

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	2	0	0
Medium	0	0	0	0
Minor / Informative	3	4	0	0

ST - Stops Transactions

Criticality	Critical
Location	UCBIHolding.sol#L15,63,67
Status	Acknowledged

Description

The contract owner has the authority to stop transactions for all users excluding the owner and the addresses listed in the `allowedAddresses` mapping. The owner may take advantage of it by setting the `locked` variable to true via the `setLocked` method. The `locked` flag is initialized to true at contract construction, which means transfers are disabled by default until the owner explicitly calls `setLocked(false)`. Additionally, the owner is able to add the pair address to the `lockedAddresses` mapping through the `lockAddress` function, which prevents the pair from sending tokens to buyers and blocks all buy operations against the pool. As a result, the contract may operate as a honeypot.

```
bool public locked = true;

function setLocked(bool _locked) external onlyOwner {(...)}

function canTransfer(address _addr) public view returns (bool) {
    if (locked) {
        if (!allowedAddresses[_addr] && _addr != owner) return false;
    } else {
        if (lockedAddresses[_addr]) return false;
    }
    return true;
}
```

Recommendation

The contract could embody a check for not allowing setting the `locked` state to true once trading has been enabled. A suggested implementation could check that the `locked flag` cannot be re-enabled after it has been initially disabled. Additionally, important contract address should always be treated as excluded from conditions that restrict the from transferring tokens.

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions. Temporary Solutions: These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue and states:

In our case, the UCBI_Banking contract is intentionally designed with a degree of centralization for economic, organizational, and regulatory reasons: Temporary wallet restrictions Our economic model requires certain lock-up periods. This allows us to:

stabilize the token market prevent excessive short-term speculation meet commitments made to our investors

Transfer limitations Transfers may be restricted because our shareholders and partners are currently operating within a centralized process. This ensures:

better control over financial flows compliance with our internal structure a gradual transition before broader openness

Contract owner role At this stage, we cannot:

remove the owner role nor renounce ownership

because governance is still centralized. As a structured company, this enables us to:

maintain operational security intervene quickly if needed manage the project in line with our roadmap

Regulatory compliance (KYC/AML) Our company operates within a regulatory framework that requires strict compliance with Know Your Customer (KYC) and Anti-Money Laundering (AML) procedures. As a result, we must retain the technical ability to:

monitor and control transactions when necessary restrict or block wallets involved in suspicious activities act in accordance with regulations related to anti-money laundering and counter-terrorism financing

These measures are essential to ensure legal compliance, protect the ecosystem, and align with the obligations imposed on us as a regulated entity. Conclusion In summary, the mechanisms identified (blacklisting, pausing transactions, centralized control) are not considered vulnerabilities in our context, but rather intentional design choices aligned with our current development phase, economic model, and regulatory obligations. That said, we remain open to evolving the architecture (e.g., multi-signature mechanisms, progressive decentralization) as our ecosystem and governance mature..

BC - Blacklists Addresses

Criticality	Critical
Location	UCBIHolding.sol#L21,58
Status	Acknowledged

Description

The contract owner has the authority to stop addresses from transactions. The owner may take advantage of it by calling the `lockAddress` function.

```
mapping(address => bool) public lockedAddresses;

function lockAddress(address _addr, bool _locked) external onlyOwner
{(...)}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue and states:

In our case, the UCBI_Banking contract is intentionally designed with a degree of centralization for economic, organizational, and regulatory reasons: Temporary wallet restrictions Our economic model requires certain lock-up periods. This allows us to:

stabilize the token market prevent excessive short-term speculation meet commitments made to our investors

Transfer limitations Transfers may be restricted because our shareholders and partners are currently operating within a centralized process. This ensures:

better control over financial flows compliance with our internal structure a gradual transition before broader openness

Contract owner role At this stage, we cannot:

remove the owner role nor renounce ownership

because governance is still centralized. As a structured company, this enables us to:

maintain operational security intervene quickly if needed manage the project in line with our roadmap

Regulatory compliance (KYC/AML) Our company operates within a regulatory framework that requires strict compliance with Know Your Customer (KYC) and Anti-Money Laundering (AML) procedures. As a result, we must retain the technical ability to:

monitor and control transactions when necessary restrict or block wallets involved in suspicious activities act in accordance with regulations related to anti-money laundering and counter-terrorism financing

These measures are essential to ensure legal compliance, protect the ecosystem, and align with the obligations imposed on us as a regulated entity. Conclusion In summary, the mechanisms identified (blacklisting, pausing transactions, centralized control) are not considered vulnerabilities in our context, but rather intentional design choices aligned with our current development phase, economic model, and regulatory obligations. That said, we remain open to evolving the architecture (e.g., multi-signature mechanisms, progressive decentralization) as our ecosystem and governance mature..

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	UCBIHolding.sol#L43,53,58,63
Status	Acknowledged

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```
function setLocked(bool _locked) external onlyOwner {(...)}

function allowAddress(address _addr, bool _allowed) external onlyOwner
{(...)}

function lockAddress(address _addr, bool _locked) external onlyOwner
{(...)}

function transferOwnership(address newOwner) external onlyOwner {(...)}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

Team Update

The team has acknowledged that this is not a security issue and states:

In our case, the UCBI_Banking contract is intentionally designed with a degree of centralization for economic, organizational, and regulatory reasons: Temporary wallet restrictions Our economic model requires certain lock-up periods. This allows us to:

stabilize the token market prevent excessive short-term speculation meet commitments made to our investors

Transfer limitations Transfers may be restricted because our shareholders and partners are currently operating within a centralized process. This ensures:

better control over financial flows compliance with our internal structure a gradual transition before broader openness

Contract owner role At this stage, we cannot:

remove the owner role nor renounce ownership

because governance is still centralized. As a structured company, this enables us to:

maintain operational security intervene quickly if needed manage the project in line with our roadmap

Regulatory compliance (KYC/AML) Our company operates within a regulatory framework that requires strict compliance with Know Your Customer (KYC) and Anti-Money Laundering (AML) procedures. As a result, we must retain the technical ability to:

monitor and control transactions when necessary restrict or block wallets involved in suspicious activities act in accordance with regulations related to anti-money laundering and counter-terrorism financing

These measures are essential to ensure legal compliance, protect the ecosystem, and align with the obligations imposed on us as a regulated entity. Conclusion In summary, the mechanisms identified (blacklisting, pausing transactions, centralized control) are not considered vulnerabilities in our context, but rather intentional design choices aligned with our current development phase, economic model, and regulatory obligations. That said, we remain open to evolving the architecture (e.g., multi-signature mechanisms, progressive decentralization) as our ecosystem and governance mature..

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	UCBIHolding.sol#L53,58,63
Status	Acknowledged

Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

```
function allowAddress(address _addr, bool _allowed) external onlyOwner
{(...)}

function lockAddress(address _addr, bool _locked) external onlyOwner
{(...)}

function setLocked(bool _locked) external onlyOwner {(...)}
```

Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

MRO - Missing Renounce Ownership

Criticality	Minor / Informative
Location	UCBIHolding.sol#LL13
Status	Acknowledged

Description

The contract implements custom ownership functionality through the owner state variable and the onlyOwner modifier but does not provide a way for the owner to renounce ownership and remove privileged control. As a result, the owner will always retain full administrative authority over sensitive functions such as locking transfers and managing address permissions, which introduces long-term centralization and trust concerns.

```
address public owner;

modifier onlyOwner() {
    require(msg.sender == owner, "Not owner");
    _;
}
```

Recommendation

The team is advised to implement a renounce ownership mechanism that allows the owner to permanently remove their privileges once they are no longer needed. This helps increase trust in the system by ensuring that administrative control can be fully relinquished.

MTEE - Missing Transfer Event Emission

Criticality	Minor / Informative
Location	UCBIHolding.sol#L32
Status	Acknowledged

Description

The contract does not emit an event when portions of the main amount are transferred during the transfer process. This lack of event emission results in decreased transparency and traceability regarding the flow of tokens, and hinders the ability of decentralized applications (dApps), such as blockchain explorers, to accurately track and analyze these transactions.

```
constructor() {
    owner = msg.sender;
    totalSupply = MAX_SUPPLY;
    balanceOf[msg.sender] = MAX_SUPPLY;
    allowedAddresses[msg.sender] = true;
}
```

Recommendation

It is advisable to incorporate the emission of detailed event logs following each asset transfer. These logs should encapsulate key transaction details, including the identities of the sender and receiver, and the quantity of assets transferred. Implementing this practice will enhance the reliability and transparency of transaction tracking systems, ensuring accurate data availability for ecosystem participants.

L02 - State Variables could be Declared Constant

Criticality	Minor / Informative
Location	UCBIHolding.sol#L6,7,8
Status	Unresolved

Description

State variables can be declared as constant using the constant keyword. This means that the value of the state variable cannot be changed after it has been set. Additionally, the constant variables decrease gas consumption of the corresponding transaction.

```
string public name = "UCBI Holding"
string public symbol = "UCBI"
uint8 public decimals = 8
```

Recommendation

Constant state variables can be useful when the contract wants to ensure that the value of a state variable cannot be changed by any function in the contract. This can be useful for storing values that are important to the contract's behavior, such as the contract's address or the maximum number of times a certain function can be called. The team is advised to add the constant keyword to state variables that never change.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	UCBIHolding.sol#L53,58,63,67
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```
address _addr
bool _allowed
bool _locked
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.
Find more information on the Solidity documentation https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	UCBIHolding.sol#L2
Status	Unresolved

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```
pragma solidity ^0.8.20;
```

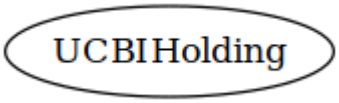
Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

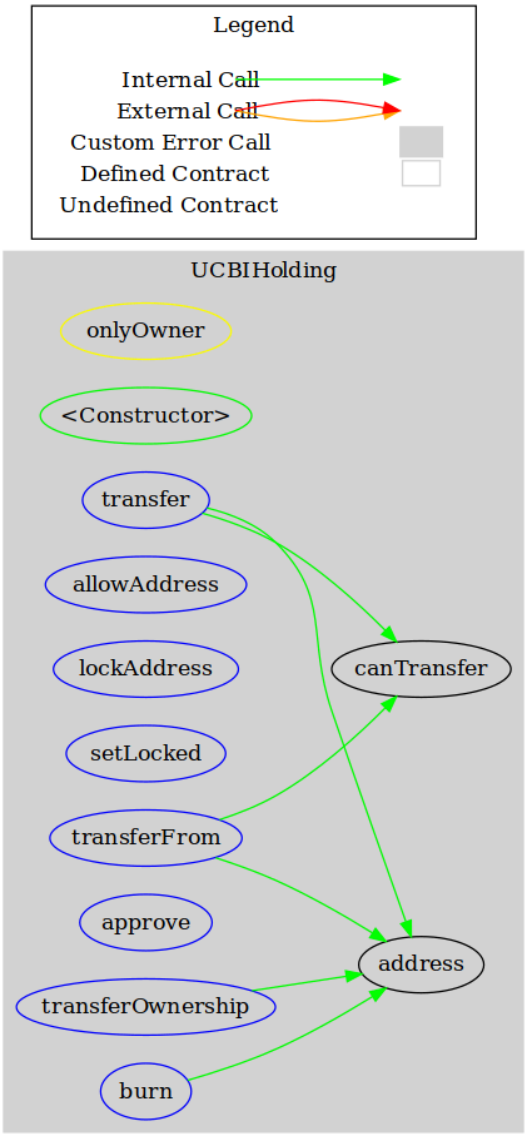
Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
UCBIHolding	Implementation			
		Public	✓	-
	transferOwnership	External	✓	onlyOwner
	allowAddress	External	✓	onlyOwner
	lockAddress	External	✓	onlyOwner
	setLocked	External	✓	onlyOwner
	canTransfer	Public		-
	transfer	External	✓	-
	approve	External	✓	-
	transferFrom	External	✓	-
	burn	External	✓	-

Inheritance Graph



Flow Graph


Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
UCBIHolding
onlyOwner
<Constructor>
transfer
allowAddress
lockAddress
setLocked
transferFrom
approve
transferOwnership
burn
canTransfer
address

Summary

UCBI_Banking contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the owner like stop transactions and massively blacklist addresses. A multi-wallet signing pattern will provide security against potential hacks. Temporarily locking the contract or renouncing ownership will eliminate all the contract threats.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io